

```
1 # Install if needed:
2 # pip install pandas scikit-learn matplotlib seaborn
3
4 import pandas as pd
5 import re
6 from sklearn.feature_extraction.text import TfidfVectorizer
7 from sklearn.cluster import KMeans
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.model_selection import train_test_split
10 from sklearn.metrics import classification_report
```

```
1 # ✅ TOP of datm.py
2
3 import streamlit as st
4 from transformers import BertTokenizer, BertForSequenceClassification
5
6 # 📌 PUT YOUR CACHED MODEL CODE HERE
7 @st.cache_resource
8 def load_model():
9     tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
10     model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
11     return tokenizer, model
12
13 tokenizer, model = load_model()
```

```
1 df = pd.read_csv("ingredients v1.csv")
2
3 # Keep only necessary columns
4 df = df[['name', 'brand', 'features.value']]
5
6 # Drop missing values
7 df = df.dropna()
8
9 # Rename for convenience
10 df.rename(columns={'features.value': 'ingredients'}, inplace=True)
11
12 # Take sample for pilot study
13 df = df.sample(200, random_state=42)
```

```
1 def clean_text(text):
2     text = str(text).lower()
```

```
3 text = re.sub(r'ingredients:', '', text)
4 text = re.sub(r'^a-zA-Z0-9, ]', '', text)
5 return text.strip()
6
7 df['clean_ingredients'] = df['ingredients'].apply(clean_text)
```

```
1 # Expanded Ingredient Dictionaries
2
3 # 🍬 Hidden Sugars
4 sugar_list = [
5     "sugar", "glucose", "fructose", "dextrose",
6     "corn syrup", "maltose", "sucrose", "high fructose corn syrup"
7 ]
8
9 # 🌴 Palm Oil
10 palm_oil_list = [
11     "palm oil", "palm kernel oil", "palm fat", "vegetable oil"
12 ]
13
14 # ⚠️ Artificial Sweeteners
15 sweeteners_list = [
16     "aspartame", "sucralose", "saccharin", "acesulfame k"
17 ]
18
19 # 🧂 High Sodium Ingredients
20 sodium_list = [
21     "sodium", "monosodium glutamate", "msg", "sodium nitrate", "sodium nitrite"
22 ]
23
24 # 🧴 Preservatives
25 preservatives_list = [
26     "sodium benzoate", "potassium sorbate", "bha", "bht"
27 ]
28
29 # 🌈 Artificial Colors
30 colors_list = [
31     "e100", "e102", "e110", "e129", "tartrazine", "caramel color"
32 ]
33
34 # 🥗 Ultra-processed ingredients
35 processed_list = [
36     "maltodextrin", "modified starch", "hydrogenated oil"
37 ]
38
39 # 🗑️ Allergens (optional)
```

```
40 allergens_list = [  
41     "soy", "gluten", "milk", "peanut", "wheat"  
42 ]
```

```
1 def detect_flags(text):  
2     return {  
3         "hidden_sugar": int(any(word in text for word in sugar_list)),  
4         "palm_oil": int(any(word in text for word in palm_oil_list)),  
5         "sweeteners": int(any(word in text for word in sweeteners_list)),  
6         "high_sodium": int(any(word in text for word in sodium_list)),  
7         "preservatives": int(any(word in text for word in preservatives_list)),  
8         "artificial_colors": int(any(word in text for word in colors_list)),  
9         "ultra_processed": int(any(word in text for word in processed_list)),  
10        "allergens": int(any(word in text for word in allergens_list))  
11    }
```

```
1 def calculate_risk(row):  
2     score = 0  
3     score += row['hidden_sugar'] * 2  
4     score += row['palm_oil'] * 2  
5     score += row['sweeteners'] * 2  
6     score += row['high_sodium'] * 2  
7     score += row['preservatives'] * 2  
8     score += row['artificial_colors'] * 2  
9     score += row['ultra_processed'] * 3  
10  
11     if score >= 7:  
12         return "High"  
13     elif score >= 4:  
14         return "Medium"  
15     else:  
16         return "Low"
```

```
1 # Apply detection to create feature columns  
2 flags_df = df['clean_ingredients'].apply(detect_flags).apply(pd.Series)  
3  
4 # Merge back into dataframe  
5 df = pd.concat([df, flags_df], axis=1)  
6  
7 # Calculate risk  
8 df['risk'] = df.apply(calculate_risk, axis=1)
```

```

1 # Convert risk labels to numbers
2 risk_map = {"Low": 0, "Medium": 1, "High": 2}
3 df['risk_label'] = df['risk'].map(risk_map)
4
5 # TF-IDF Vectorization
6 vectorizer = TfidfVectorizer(max_features=500)
7 X = vectorizer.fit_transform(df['clean_ingredients'])
8 y = df['risk_label']
9
10 # Train-test split
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
12
13 # Train model
14 model = LogisticRegression()
15 model.fit(X_train, y_train)
16
17 # Predictions
18 y_pred = model.predict(X_test)
19
20 # Evaluation
21 print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0	0.72	0.96	0.83	27
1	0.75	0.33	0.46	9
2	0.00	0.00	0.00	4
accuracy			0.72	40
macro avg	0.49	0.43	0.43	40
weighted avg	0.66	0.72	0.66	40

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in labels [1, 2] with no predicted samples. Use 'zero_division' parameter to control this warning.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in labels [1, 2] with no predicted samples. Use 'zero_division' parameter to control this warning.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in labels [1, 2] with no predicted samples. Use 'zero_division' parameter to control this warning.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```

```

1 kmeans = KMeans(n_clusters=3, random_state=42)
2 df['cluster'] = kmeans.fit_predict(X)
3
4 print(df[['name', 'risk', 'cluster']].head())

```

	name	risk	cluster
8993	Bigelow Lemon Lift Black Tea - 20 CT	Low	2
3264	Aqua Star Breaded Butterfly Crunchy Shrimp, 9 oz	Medium	1
9581	Marshall's Creek Spices Marjoram Seasoning, 3 O...	Low	2
7308	Gold Emblem Abound Organic Maple Syrup, 8 OZ	Low	2
2332	Green Mountain Coffee® Flavored Rain Forest N...	Low	2

```

1 print("Risk Distribution:\n", df['risk'].value_counts())
2
3 print("\nHidden Sugar %:", df['hidden_sugar'].mean() * 100)
4 print("Palm Oil %:", df['palm_oil'].mean() * 100)
5 print("Preservatives %:", df['preservatives'].mean() * 100)
6 print("Artificial Colors %:", df['artificial_colors'].mean() * 100)

```

```

Risk Distribution:
risk
Low      128
Medium   48
High     24
Name: count, dtype: int64

Hidden Sugar %: 49.0
Palm Oil %: 13.5
Preservatives %: 8.5
Artificial Colors %: 6.0

```

```
1 df.to_csv("processed_food_analysis.csv", index=False)
```

```

1 # pip install transformers torch datasets scikit-learn
2
3 from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
4 import torch
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score, f1_score

```

```

1 # Use only rows where risk_label exists
2 df_bert = df[['clean_ingredients', 'risk_label']].dropna()
3
4 # Train-test split
5 X_train_text, X_test_text, y_train, y_test = train_test_split(
6     df_bert['clean_ingredients'],
7     df_bert['risk_label'],
8     test_size=0.2,

```

```

9     random_state=42
10 )

```

```

1 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
2
3 def tokenize(texts):
4     return tokenizer(
5         texts.tolist(),
6         padding=True,
7         truncation=True,
8         max_length=128
9     )
10
11 train_encodings = tokenize(X_train_text)
12 test_encodings = tokenize(X_test_text)

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.34kB/s]

vocab.txt: 232k/? [00:00<00:00, 2.05MB/s]

tokenizer.json: 466k/? [00:00<00:00, 3.31MB/s]

```

1 class FoodDataset(torch.utils.data.Dataset):
2     def __init__(self, encodings, labels):
3         self.encodings = encodings
4         self.labels = labels.tolist()
5
6     def __getitem__(self, idx):
7         item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
8         item['labels'] = torch.tensor(self.labels[idx])
9         return item
10
11     def __len__(self):
12         return len(self.labels)
13

```

```
14 train_dataset = FoodDataset(train_encodings, y_train)
15 test_dataset = FoodDataset(test_encodings, y_test)
```

```
1 model = BertForSequenceClassification.from_pretrained(
2     'bert-base-uncased',
3     num_labels=3
4 )
```

config.json: 100% 570/570 [00:00<00:00, 52.8kB/s]

model.safetensors: 100% 440M/440M [00:07<00:00, 56.3MB/s]

Loading weights: 100% 199/199 [00:00<00:00, 437.53it/s, Materializing param=bert.pooler.dense.weight]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task

```
1 training_args = TrainingArguments(
2     output_dir='./results',
3     num_train_epochs=1,
4     per_device_train_batch_size=8,
5     per_device_eval_batch_size=8,
6     logging_dir='./logs'
7 )
```

`logging\_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD\_LOGGING\_DIR` instead.

```
1 def compute_metrics(pred):
2     labels = pred.label_ids
3     preds = pred.predictions.argmax(-1)
4
```

```

5     return {
6         "accuracy": accuracy_score(labels, preds),
7         "f1": f1_score(labels, preds, average='weighted')
8     }

```

```

1 trainer = Trainer(
2     model=model,
3     args=training_args,
4     train_dataset=train_dataset,
5     eval_dataset=test_dataset,
6     compute_metrics=compute_metrics
7 )

```

```
1 trainer.train()
```

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true
super().__init__(loader)

 [20/20 03:33, Epoch 1/1]

Step Training Loss

Writing model shards: 100%

1/1 [00:04<00:00, 4.93s/it]

TrainOutput(global_step=20, training_loss=0.9557243347167969, metrics={'train_runtime': 230.9697, 'train_samples_per_second': 0.693, 'train_steps_per_second': 0.087, 'total_flos': 10524536709120.0, 'train_loss': 0.9557243347167969, 'epoch': 1.0})

```

1 results = trainer.evaluate()
2 print(results)

```

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true
super().__init__(loader)

 [5/5 00:12]

{'eval_loss': 0.8198491930961609, 'eval_accuracy': 0.675, 'eval_f1': 0.5440298507462686, 'eval_runtime': 14.8261, 'eval_sample

```

1
2 import pytesseract
3 from PIL import Image
4
5 img = Image.open("test.png") # make sure file exists
6 text = pytesseract.image_to_string(img)
7
8 print(text)

```

```
Requirement already satisfied: pytesseract in /usr/local/lib/python3.12/dist-packages (0.3.13)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from pytesseract) (26.1)
Requirement already satisfied: Pillow>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from pytesseract) (11.3.0)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tesseract-ocr is already the newest version (4.1.1-2.1build1).
0 upgraded, 0 newly installed, 0 to remove and 61 not upgraded.
```

INGREDIENTS: ENRICHED FLOUR (WHEAT FLOUR, NIACIN, REDUCED IRON, THIAMIN MONONITRATE [VITAMIN B₁], RIBOFLAVIN a Be FOLIC. ACID), CORN SYRUP. 'AND PALM OIL

FOR FRESHNESS), CORN SYRUP SOLIDS, DEXTROSE, HIGH FRUCTOSE CORN SYRUP FRUCTOSE, GLYCERIN, CONTAINS 2% OR LESS OF COCOA (PROCESSED WITH ALKALI), POLYDEXTROSE, MODIFIED CORN STARCH, SALT, DRIED CREAM, CALCIUM CARBONATE, CORNSTARCH, LEAVENING (BAKING SODA, SODIUM ACID PYROPHOSPHATE, MONOCALCIUM PHOSPHATE, CALCIUM SULFATE), DISTILLED MONOGLYCERIDES, HYDROGENATED PALM KERNEL OIL, SODIUM STEAROYL LACTYLATE, GELATIN, COLOR ADDED, SOY LECITHIN, OATEM, NATURAL AND ARTIFICIAL FLAVOR, VANILLA EXTRACT, CARNAUBA WAX, XANTHAN GUM, VITAMIN A PALMITATE, YELLOW #5 LAKE. RED #40 LAKE, CARAMEL COLOR, NIACINAMIDE, BLUE #2 LAKE, REDUCED IRON, YELLOW #6 LAKE, PYRIDOXINE HYDROCHLORIDE (VITAMIN B₆), RIBOFLAVIN (VITAMIN B₂), THIAMIN HYDROCHLORIDE (VITAMIN B₁), CITRIC ACID, FOLIC ACID, RED #40, YELLOW #5, YELLOW #8, BLUE #2, BLUE #1.

```
1 def analyse_ingredients(text):
2     clean = clean_text(text)
3     flags = detect_flags(clean)
4
5     # Convert to row-like structure
6     flags_row = pd.Series(flags)
7
8     risk = calculate_risk(flags_row)
9
10    # Count bad ingredients
```

```

11     bad_count = sum(flags.values())
12
13     return risk, bad_count, flags

```

```

1 import streamlit as st
2 from PIL import Image
3 import pandas as pd
4
5 st.title("🍌 Food Label Decoder")
6
7 st.write("Upload a food label image to analyse harmful ingredients.")
8
9 uploaded_file = st.file_uploader("Upload Image", type=["jpg", "png"])
10
11 if uploaded_file:
12     image = Image.open(uploaded_file)
13     st.image(image, caption="Uploaded Image", use_column_width=True)
14
15     # Extract text
16     text = extract_text_from_image(image)
17
18     st.subheader("Extracted Text")
19     st.write(text)
20
21     # Analyse
22     risk, bad_count, flags = analyse_ingredients(text)
23
24     st.subheader("Analysis Result")
25     st.write(f"Risk Level: **{risk}**")
26     st.write(f"Number of harmful ingredient categories detected: **{bad_count}**")
27
28     st.subheader("Detected Issues")
29     for k, v in flags.items():
30         if v == 1:
31             st.write(f"⚠️ {k.replace('_', ' ').title()}")

```

2026-05-02 18:30:59.535 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
 2026-05-02 18:30:59.895

Warning: to view this Streamlit app on a browser, run it with the following command:

```

streamlit run /usr/local/lib/python3.12/dist-packages/colab_kernel_launcher.py [ARGUMENTS]
2026-05-02 18:30:59.898 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.900 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.905 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.

```

```
2026-05-02 18:30:59.906 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.909 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.914 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.915 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.918 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.921 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.927 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-05-02 18:30:59.932 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
1 from pyngrok import ngrok
2
3 ngrok.set_auth_token("3DAtHAMf3ZEWlSaJKZqpAxlkYzD_3XYqih2EiYikFBNXJun5fN")
```

```
1 Start coding or generate with AI.
```